

How to Build High Throughput Systems



BYTEBYTEGO

MAR 05, 2026 • PAID



142

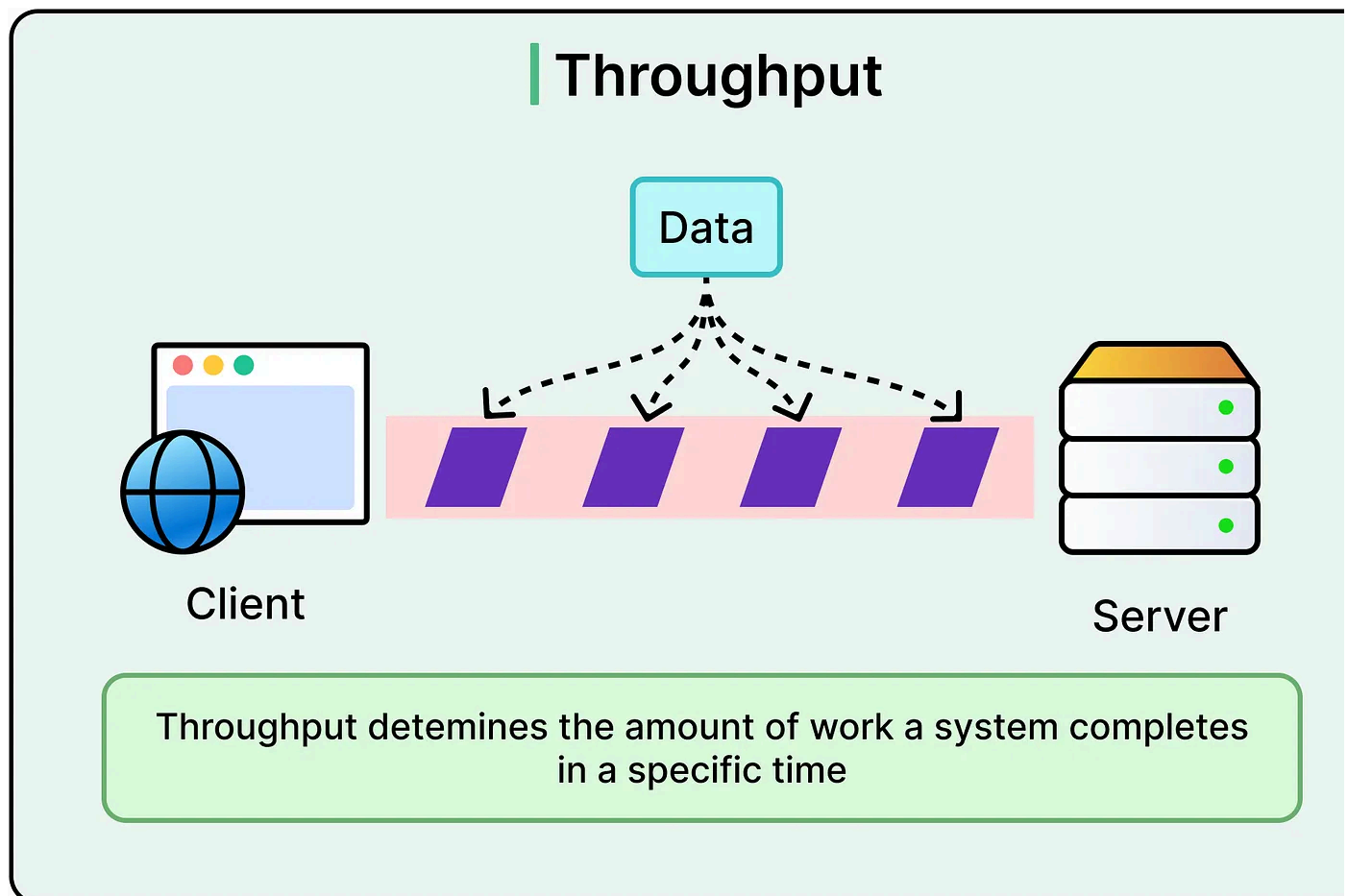


8

Share

When we build software systems, one of the most important goals is making sure that it can handle large amounts of work efficiently.

High-throughput systems are capable of processing vast quantities of data or transactions in a given timeframe. Throughput refers to the amount of work a system completes in a specific time period. For example, a web server might process 10K requests per second, or a database might handle 50K transactions per minute. The higher the throughput, the more work gets done in the same amount of time.



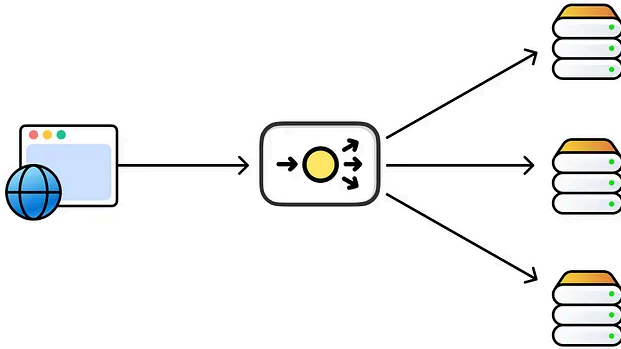
Throughput is different from latency. Latency measures how long it takes to complete a single operation, from start to finish. While throughput measures the volume of operations the system handles over time. For example, a system can have low latency but low throughput if it processes each request quickly but can only handle a few at once. Conversely, a system might have high throughput but high latency if it processes many requests simultaneously, but each request takes longer to complete.

There is often a tradeoff between these two metrics. When we batch multiple operations together, we increase throughput because the system processes many items at once. However, this batching introduces waiting time for individual operations, which increases latency. Similarly, processing every request immediately reduces latency but may limit throughput if the system becomes overwhelmed.

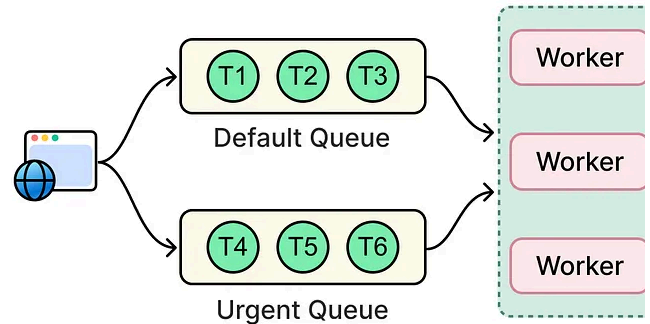
In this article, we will go through the fundamental concepts and practical strategies for building systems that can handle high volumes of work without breaking down under pressure.

High-Throughput Strategies

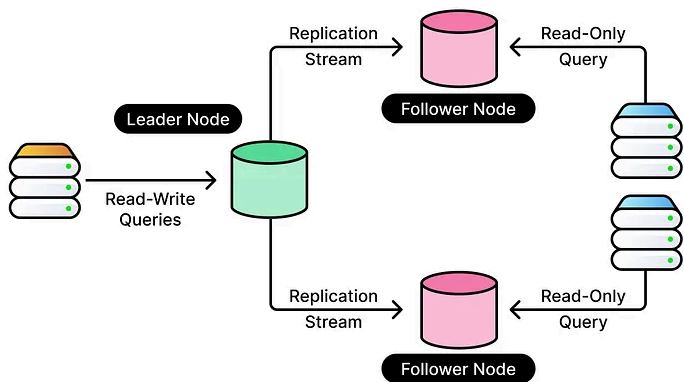
Load Balancing



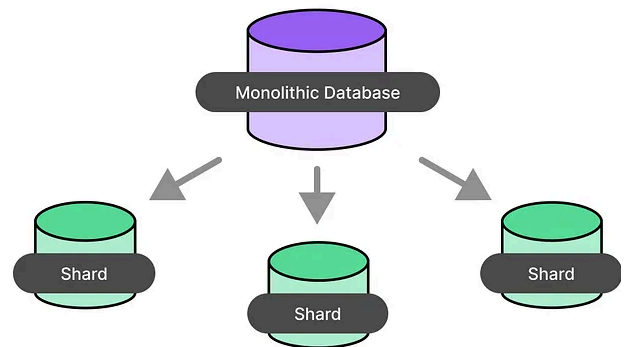
Async Task Execution with Queues



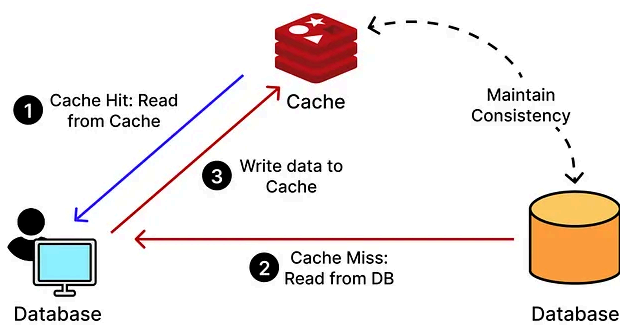
Database Replication



Database Sharding



Caching



Database Indexing

Email	Pointer			
alice@example.com	2	1	John	alice@example.com
bob@example.com	3	2	Alice	alice@example.com
jane@example.com	5	3	Bob	bob@example.com
john@example.com	1	4	Mark	mark@example.com
mark@example.com	4	5	Jane	jane@example.com

Why High Throughput Matters

High throughput determines how well our system scales to meet demand.

A system with higher throughput accommodates more users, handles more transactions, and processes more data without requiring additional resources. During peak traffic events like online sales, high-throughput systems process thousands of orders per second while maintaining responsiveness. High throughput also reduces costs because we can do more work with the same infrastructure, directly impacting operational efficiency and business outcomes.

Several factors determine system throughput:

- Hardware capabilities form the foundation. For example, modern CPUs with multiple cores allow parallel operations, significantly increasing throughput.
- Sufficient memory prevents swapping to disk, which would drastically reduce performance.
- Faster storage, like solid-state drives, performs many more operations per second than traditional hard drives.
- Network capacity plays a major role in distributed systems. Bandwidth determines the maximum data transmission rate. When networks become saturated, throughput bottlenecks occur regardless of server speed. Network congestion from simultaneous traffic causes delays and packet loss that reduce throughput.
- Software design significantly affects throughput. Efficient code uses fewer CPU cycles, leaving capacity for additional operations.

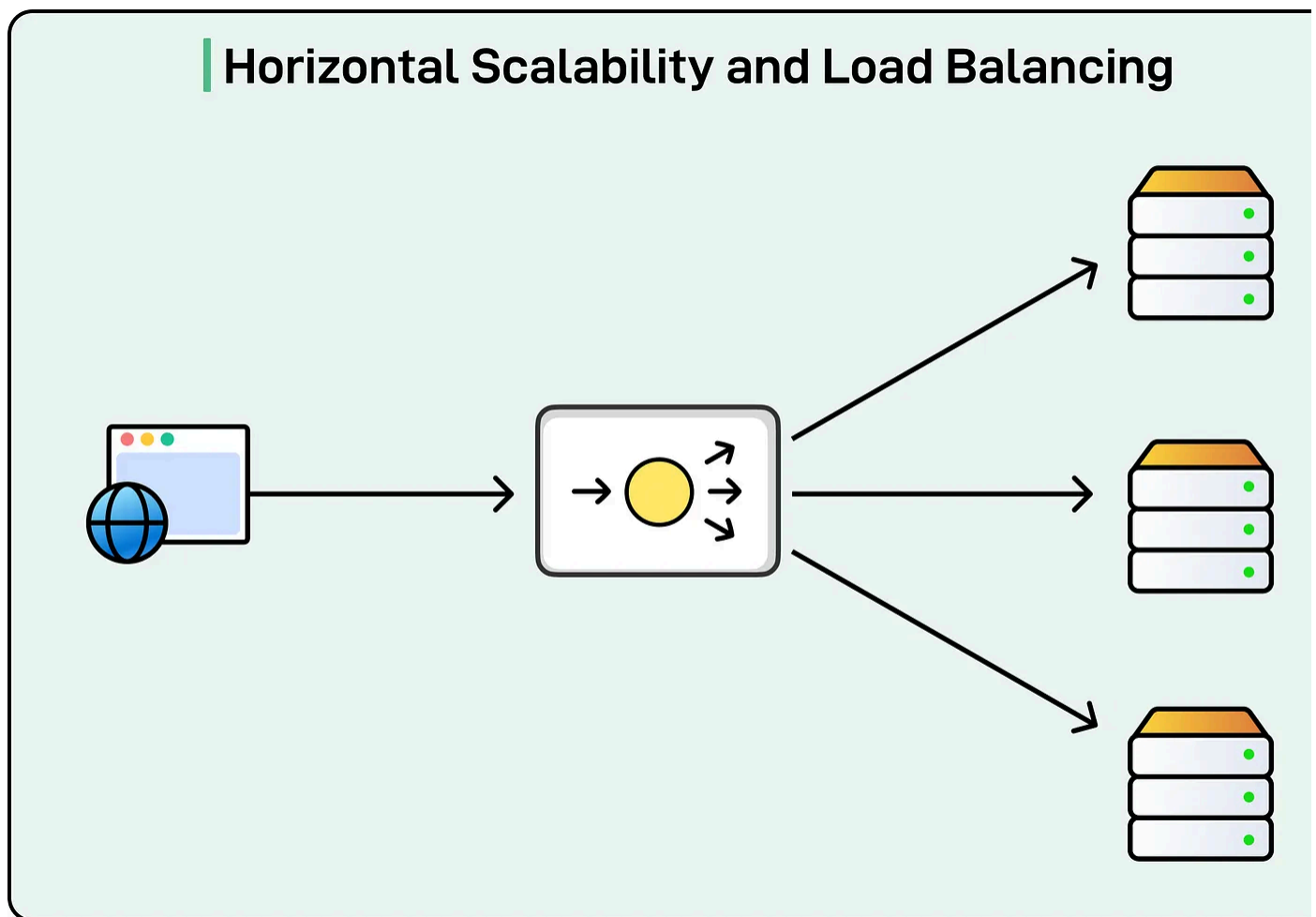
Building High Throughput Systems: Core Strategies

Now that we understand the fundamentals, let us explore the key strategies for building systems that achieve high throughput.

Horizontal Scaling and Distribution

Distributing work across multiple machines is one of the most powerful approaches to increasing throughput. Instead of relying on one server, we spread the load across many servers working together. This horizontal scaling allows us to multiply capacity. If one server processes 1,000 requests per second, ten servers can theoretically handle 10,000 requests per second.

See the diagram below:



Effective distribution requires intelligent data and request partitioning. We might divide users by geographic region or partition data by customer ID, ensuring requests for a particular customer go to the same server.

The key is enabling servers to work independently, minimizing coordination overhead. Load balancers distribute incoming requests evenly, preventing any single server from becoming overwhelmed while others sit idle.

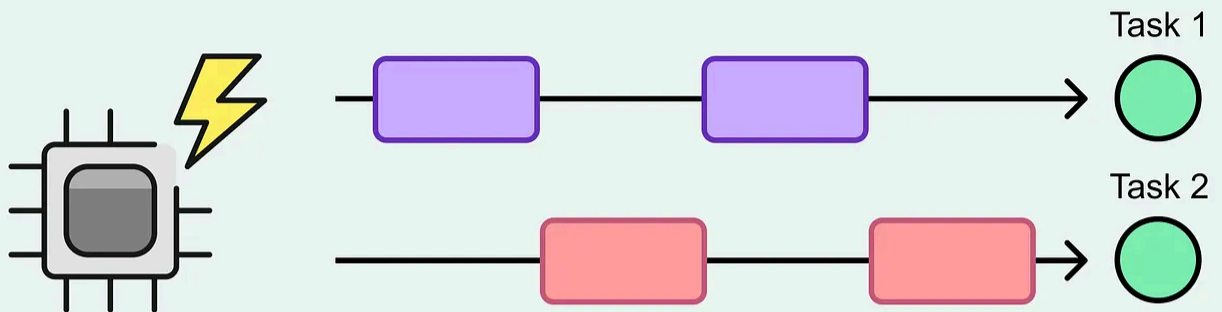
Leveraging Concurrency and Parallelism

High-throughput systems must handle many operations simultaneously through concurrency or parallelism. Concurrency manages multiple tasks in progress at the same time, even if not executing at the same instant. Parallelism actually executes multiple tasks simultaneously, requiring multiple CPU cores or machines.

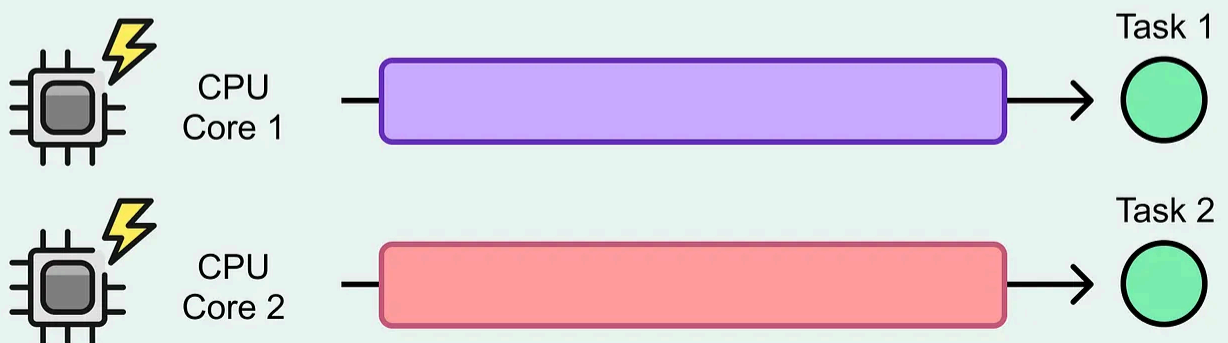
See the diagram below that shows the difference between the two approaches:

Concurrency and Parallelism

Concurrency



Parallelism



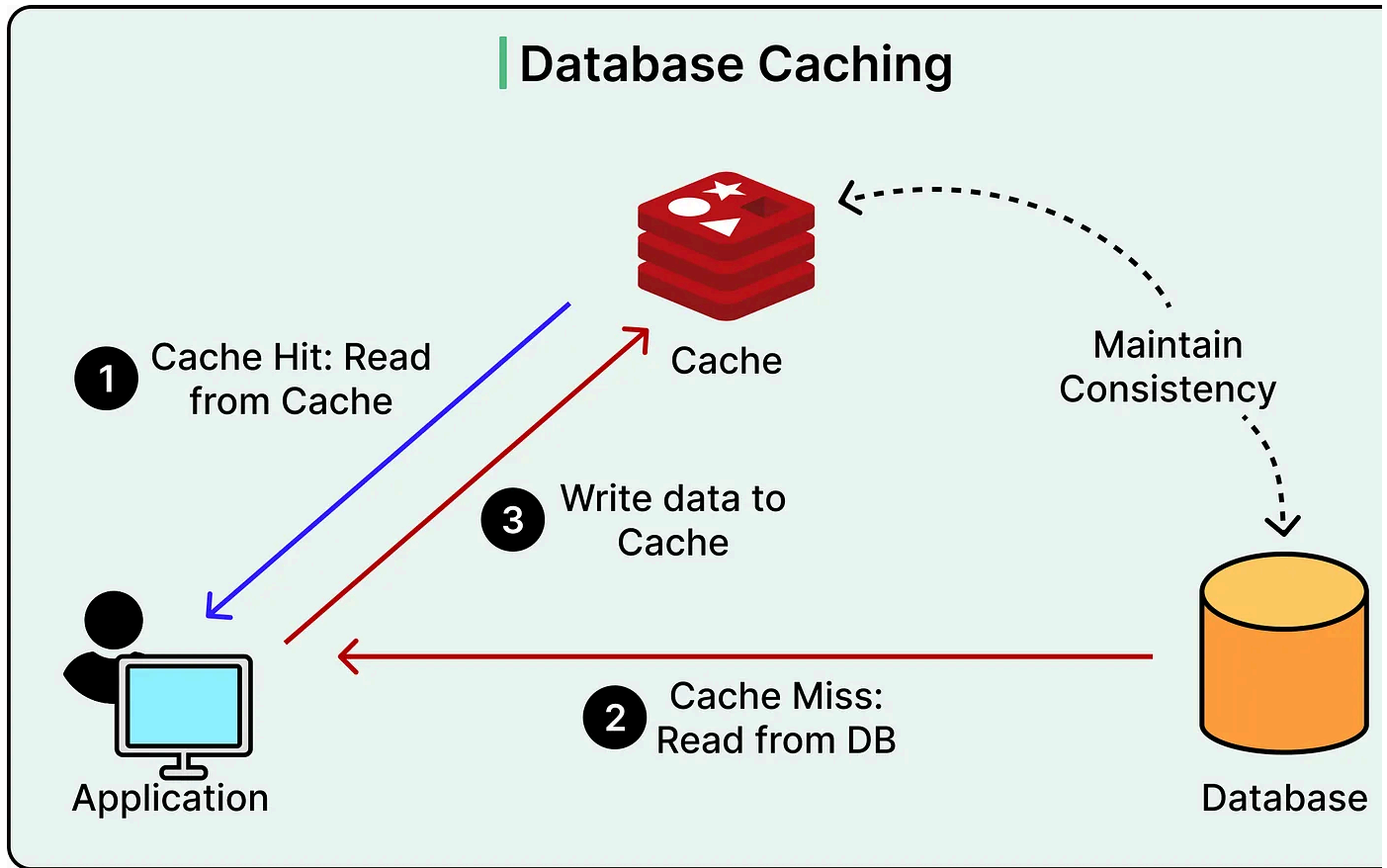
We use threads, processes, or asynchronous programming to implement concurrency. Web servers spawn multiple threads to handle requests simultaneously, preventing idle time while waiting for database queries or network calls. Asynchronous programming is particularly effective for I/O-bound operations. Instead of blocking while waiting, async systems initiate operations and switch to other work, resuming when operations complete.

However, we must avoid resource contention. When multiple threads access the same data simultaneously, they can block each other, negating the benefits of parallelism. We need to minimize shared state and use techniques like fine-grained locking or lock-free data structures to reduce contention.

Optimizing Network and I/O Operations

Network and disk I/O often bottleneck high-throughput systems. Upgrading network hardware provides immediate benefits. Moving from 1 to 10 gigabit connections increases data transfer rates tenfold. We also reduce network overhead by optimizing communication protocols through persistent connections and batching small requests into larger ones.

Caching is a powerful mechanism for reducing I/O load. Storing frequently accessed data in memory avoids repeatedly reading from disk or remote servers. Content delivery networks cache static assets close to users, reducing both latency and origin server load. In-memory caches like Redis can store database query results, eliminating expensive repeated queries.



For disk I/O, batching write operations improves throughput. Accumulating multiple writes and flushing them together is particularly effective for databases and logging systems, where sequential writes are much faster than many small random writes.

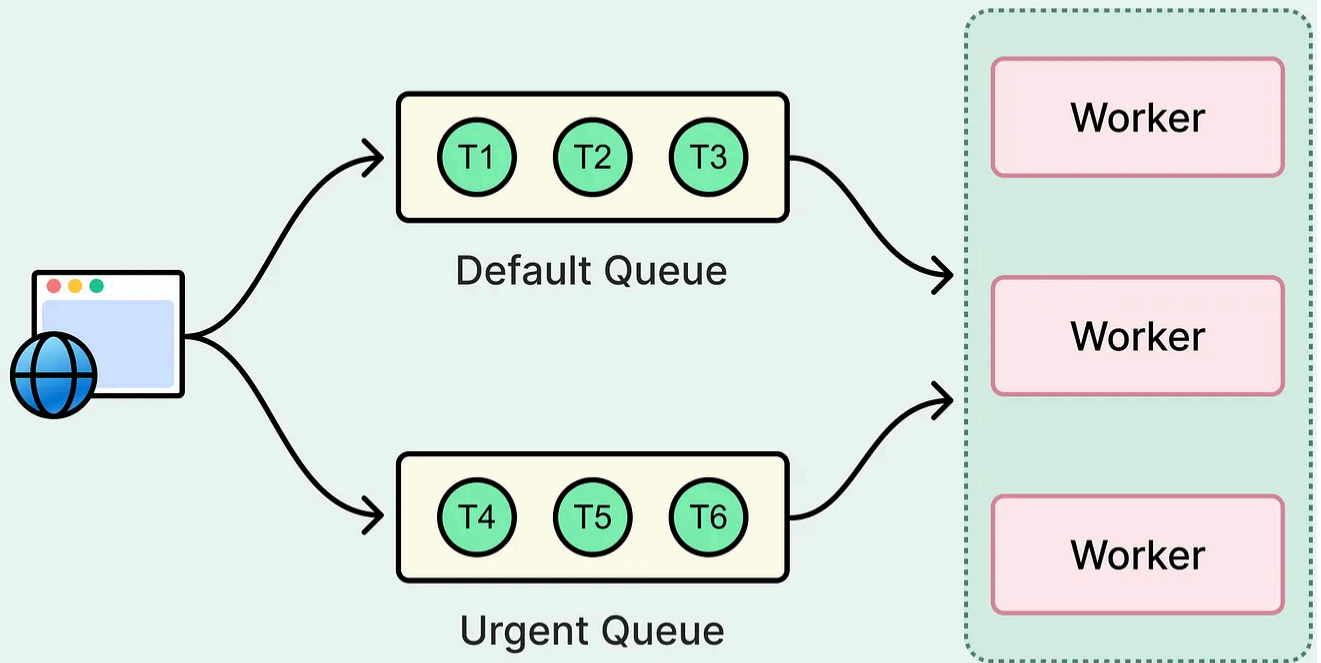
Using Asynchronous Processing and Queues

Decoupling system components using message queues and asynchronous processing significantly improves throughput.

Message queues buffer between different parts of our system. Front-end components quickly acknowledge requests, place them in queues, and move to the next request. Background workers pull items from queues and process them at their own pace.

See the diagram below:

Asynchronous Task Execution With Queues



Consider an image processing system. Users upload images quickly, but resizing and optimizing take time. With a queue, uploads are accepted immediately and queued for processing. Background workers handle images continuously, allowing the system to accept many uploads per second. This approach also provides resilience. If workers are temporarily overloaded, uploads still succeed because they are queued, and workers process the backlog when capacity is available.

High Throughput in Database Systems

Databases are often the bottleneck in high-throughput systems, so understanding how to optimize database performance is crucial.

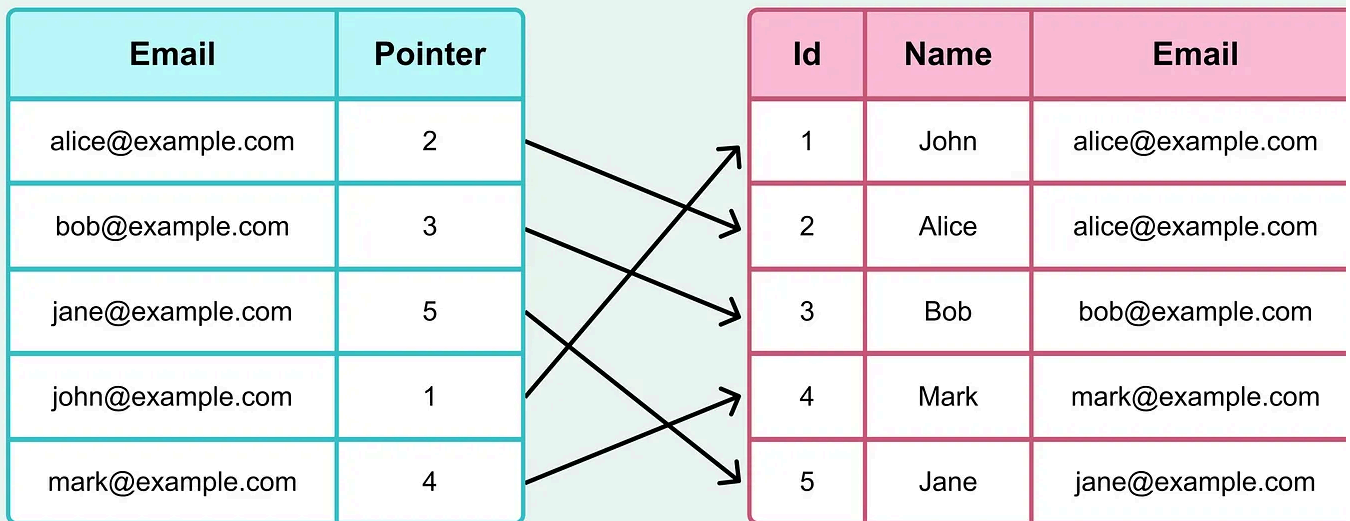
Transaction Processing Systems

Online transaction processing (OLTP) systems handle many small and quick operations. These databases power e-commerce sites, banking applications, and systems where users expect immediate responses. OLTP databases prioritize handling many concurrent transactions per second.

To achieve high throughput, we optimize for concurrent access.

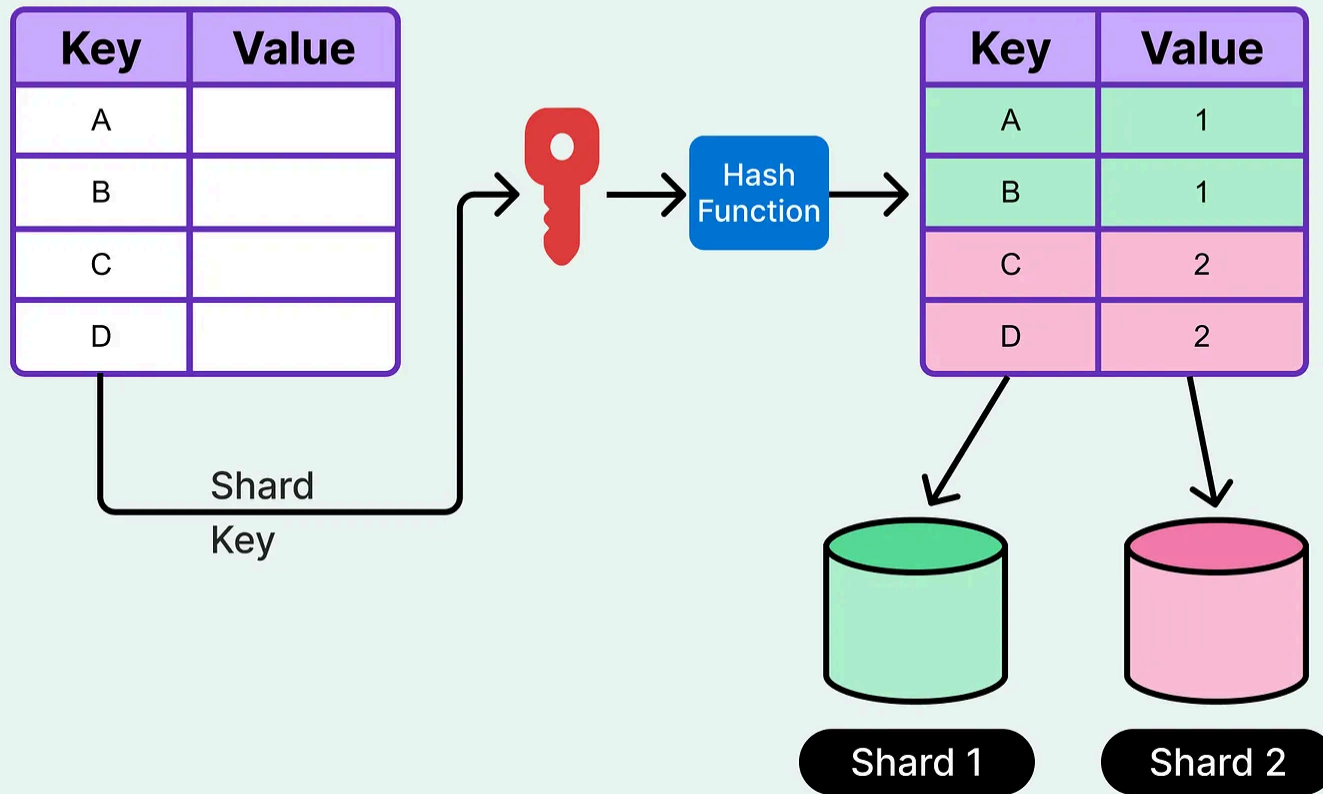
Proper indexing ensures queries find data quickly without scanning entire tables. We minimize lock contention by keeping transactions short and accessing only necessary data.

Database Index Example



Database sharding distributes data across multiple servers. Each shard contains a different subset, and applications route queries to the appropriate shard, allowing multiple servers to handle queries in parallel.

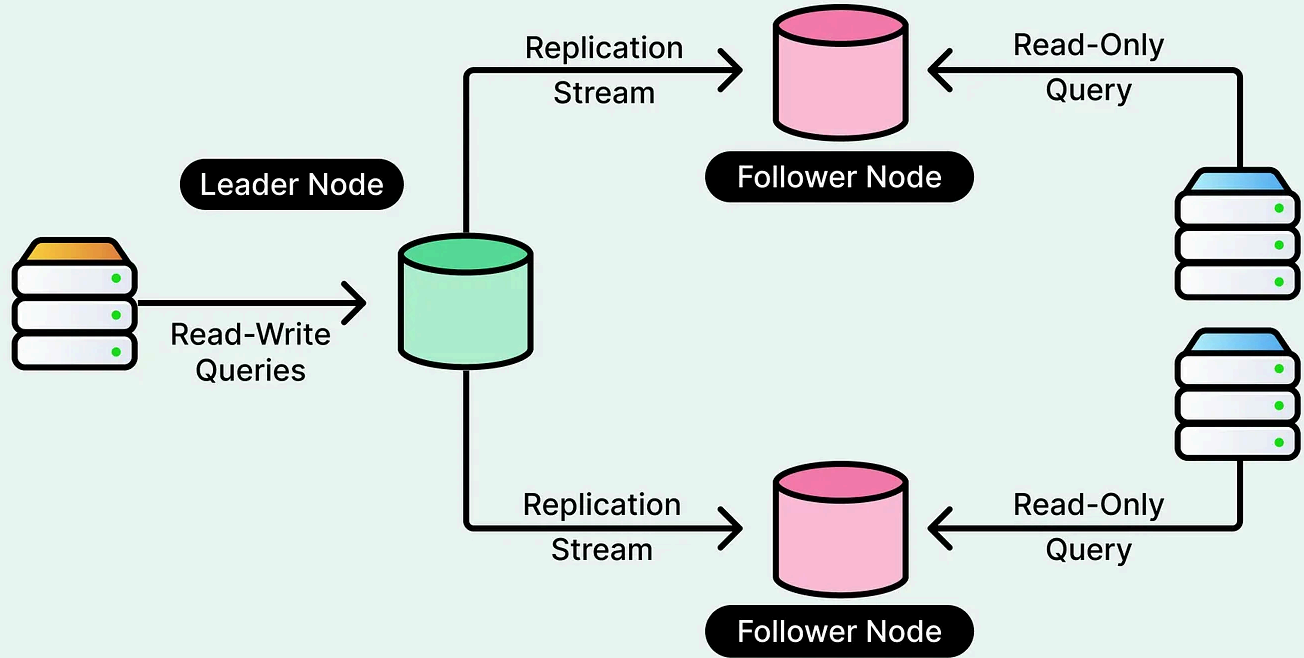
Database Sharding



Read replicas increase throughput by creating read-only database copies. Write operations go to the primary database, which replicates changes to read replicas. Read queries are distributed across all replicas, increasing read throughput for applications where reads vastly outnumber writes.

See the diagram below:

Database Replication



Analytical Processing Systems

Online analytical processing (OLAP) systems handle complex queries scanning large datasets. These systems power business intelligence dashboards, data warehouses, and analytics platforms. OLAP optimizes for data scanning throughput rather than transaction throughput.

Columnar storage formats are particularly effective for such a use case because they store column values together, allowing queries to read only needed columns. This reduces the data read from disk and increases query throughput. Parallel query execution divides large queries across multiple processors or machines. Instead of one processor scanning all data, many processors each scan a portion and combine results. Modern data warehouses automatically parallelize queries across many nodes, achieving high data processing throughput.

Understanding Tradeoffs

Building high-throughput systems requires making tradeoffs based on specific requirements.

- The throughput versus latency tradeoff is fundamental. As mentioned, techniques for increasing throughput often increase individual operation latency. Batching requests improves throughput but means each request waits for a batch to accumulate.
- In distributed systems, we also face tradeoffs between consistency and throughput. Ensuring all data copies are synchronized requires coordination between servers, reducing throughput. Many high-throughput systems accept eventual consistency, where servers might temporarily have different data views. This allows independent operation without waiting for confirmation from other servers, dramatically increasing throughput. Social media platforms often use eventual consistency, quickly accepting writes and gradually propagating them without requiring every server to coordinate on every write.
- Complexity is another tradeoff. Simple systems are easier to understand, maintain, and debug. As we add caching layers, message queues, sharding, and other optimizations, our system becomes more complex. We must balance higher throughput benefits against increased complexity costs. Sometimes, a simpler system handling less throughput is better if it meets requirements and is easier to operate.

Practical Steps for Building High Throughput Systems

When we start building a high-throughput system, following a structured approach helps us avoid common pitfalls.

- Start by measuring current throughput and identifying bottlenecks. We cannot optimize what we do not measure. Use profiling tools to understand where our

system spends time and which components limit throughput.

- Choose the right hardware for our workload. CPU-bound applications benefit from faster processors with more cores. I/O-bound applications need fast storage and network connections. Memory-intensive applications require sufficient RAM to avoid swapping.
- Design for horizontal scalability from the start. Even if we initially deploy on a single server, architecting our system to support distribution makes it easier to scale later when throughput requirements grow.
- Implement proper concurrency in the code. Use thread pools, async I/O, and other concurrency primitives appropriate for the programming language and framework. Ensure that concurrent operations do not block each other unnecessarily.
- Add caching at multiple levels. Cache database queries, API responses, and static assets. Use in-memory caches for frequently accessed data and content delivery networks for static files.
- Optimize database access patterns. Add indexes for common queries, minimize transaction scope, and consider denormalization for read-heavy workloads. Use read replicas to distribute query load.
- Use message queues to decouple components and handle bursts of traffic. This prevents temporary spikes in load from overwhelming the system and allows different components to scale independently.
- Monitor system performance continuously. Track key metrics like requests per second, error rates, and resource utilization. Set up alerts for when throughput drops below acceptable levels.
- Load test the system before deploying to production. Simulate realistic workloads to verify that our system meets throughput requirements and to identify potential issues under heavy load.

Summary

In this article, we've looked at high-throughput and the various strategies to build such systems in detail. Here are the key learning points in brief:

- Throughput measures the volume of work completed per unit time, while latency measures how long individual operations take, and we often need to balance the two metrics in system design.
- High-throughput systems can handle more users and process more data with the same infrastructure, leading to better scalability and lower costs.
- Hardware capabilities, including CPU power, memory, and storage speed, form the foundation of system throughput and must be appropriately sized for the workload.
- Network capacity and bandwidth determine maximum data transfer rates in distributed systems and can become bottlenecks if not properly provisioned.
- Horizontal scaling distributes work across multiple machines and is one of the most effective ways to increase throughput beyond the limits of a single server.
- Concurrency and parallelism allow systems to handle many operations simultaneously, keeping resources busy and increasing overall throughput.
- Asynchronous processing and message queues decouple system components, allowing them to operate independently and handle bursts of traffic without blocking.
- Caching reduces repeated work by storing frequently accessed data in memory, dramatically improving throughput for read-heavy workloads.
- OLTP databases optimize for many small, quick transactions while OLAP systems optimize for processing large volumes of data in complex queries.
- Database techniques like sharding, read replicas, and proper indexing can multiply transaction throughput without requiring more powerful hardware.

- Eventual consistency allows distributed systems to achieve higher throughput by reducing coordination overhead, though it means different servers may temporarily have different views of data.
- Building high-throughput systems requires measuring performance, identifying bottlenecks, and making informed tradeoffs between throughput, latency, consistency, and complexity.



142 Likes • 8 Restacks

Discussion about this post

Comments Restacks



Write a comment...

© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture